Introduction to Spec-Driven Development (SDD)

Building Better Software in the AI Era with GitHub Spec Kit

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation – 2026

Chapter 0

Today's Agenda

Chapter	Theme	Level	Time
Chapter 1	General – SDD Overview & Spec Kit	All	37 min
Chapter 2	Beginner – SDD Workflow Fundamentals	★	15 min
Chapter 3	Intermediate – Server-Side & API Contracts	★★	12 min
Chapter 4	Intermediate–Advanced – Real-World Complexity	★★★	12 min
Chapter 5	Advanced – Distributed Systems & Failure Models	★★★★	15 min
Chapter 6	Summary & Next Steps	All	5 min

Chapter 0

Who Is This Presentation For?

- **Beginners** 🟢
  "I let AI write code, but things keep breaking..."
  → Chapters 1–2: Learn SDD fundamentals and the workflow
- **Intermediate** 🟡
  "How should I design API contracts? What about external services?"
  → Chapters 3–4: Server-side and real-world complexity
- **Advanced** 🔴
  "How do I write specs for distributed systems? What about failure models?"
  → Chapter 5: Sagas, consistency, and failure path specification

Chapter 0

Prerequisites by Chapter

Chapter	Prerequisites	Tech Stack
Chapter 1	None	–
Chapter 2	HTML/CSS/JS basics	Vanilla JS + localStorage
Chapter 3	Node.js fundamentals	Express + SQLite
Chapter 4	External API experience	Stripe / OIDC / Terraform
Chapter 5	Distributed systems fundamentals	WebSocket / Saga / CRDT

Start from any level. Recommended: everyone begins with Chapter 1.

Chapter 0

Hands-On Environment

Option 1: GitHub Codespaces (Recommended)

Repository **Code** → **Codespaces** → **New codespace** – start instantly

Option 2: Local Setup

```
# Install the Spec Kit CLI
uv tool install specify-cli \
  --from git+https://github.com/github/spec-kit.git

# Initialize your project
specify init my-project --ai copilot
cd my-project && code .
```

Dev Container users: `specify` CLI is pre-installed

Chapter 1: General — SDD Overview & Spec Kit

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation

Section 1.1: The Problem

What Is Vibe Coding?

"Giving AI a rough idea and accepting whatever it produces."

A typical pattern

You: "Build me a chat app"

AI **silently decides**:

- Database design (MongoDB? PostgreSQL? Firebase?)
- Authentication model (JWT? Sessions? None?)
- Permission model (Admin? Roles? Everyone equal?)
- Error handling strategy (Silent? Dialog? Retry?)

You don't know about these choices. AI decided for you.

Section 1.1: The Problem

The 4 Failure Patterns of Vibe Coding

1. 🔓 AI decides permission rules without asking

"Can users vote on their own posts?" → AI decides silently

2. 🦠 Edge cases become production bugs

`0.1 + 0.2 = 0.30000000000000004` → billing bug

3. 🏗️ Over-engineered tech stack

"TODO app" → Kubernetes + Redis + GraphQL + microservices

4. 🔍 Intent buried in code — unmaintainable

3 months later: "Why this implementation?" → "AI decided. Reason unknown."

Section 1.1: The Problem

"Build Me an App" – The Results

Aspect	Vibe Coding	SDD
Input	"Build me a chat app"	Structured specification
DB Design	AI guesses	Defined in spec
Permission Model	AI guesses	Explicit matrix
Edge Cases	Discovered in production	Caught in Clarify
Output Quality	🎲 Unpredictable	✅ Predictable
Reproducibility	❌ Different every time	✅ Same spec → same output

Same prompt, dramatically different AI output quality when specs exist

Section 1.1: The Problem

AI Works Dramatically Better with Structure

❌ Without structure (Vibe Coding)

```
Vague prompt → AI guesses → Unpredictable output → Bugs
```

✅ With structure (SDD)

```
Structured spec → AI follows → Predictable, high-quality output
```

AI is an excellent implementer, but not an excellent designer.
Design is your job. Implementation is AI's job.

Section 1.1: The Problem

Question: What's Your Development Flow?

When you have AI write code...

- Do you write a **specification** before handing it to AI?
- Do you explicitly define **permission rules**?
- Do you identify **edge cases** beforehand?
- Can you **explain** why AI produced a particular output?

If even one answer is "No," you're **vibe coding**.
SDD changes that.

Section 1.1: The Problem

Why SDD Matters NOW – 3 Trends

1. 🤖 AI Capabilities Have Reached a Threshold

AI can now **reliably** generate working code from natural-language specs.
SDD gives AI's generation power **direction through structure**.

2. 📈 Software Complexity Grows Exponentially

Modern systems integrate dozens of services, frameworks, and dependencies.
Maintaining alignment without specs is **practically impossible**.

3. ⚡ Pace of Change Accelerates

Pivoting is the norm, not the exception. Requirement change = spec update → regenerate.
SDD makes pivoting a **systematic regeneration**, not a full rewrite.

Section 1.2: SDD Principles & the 6 Phases

What Is Spec-Driven Development?

> **"Write structured specifications first, then AI follows them – producing predictable, high-quality output every time."**

SDD's Key Message

- The spec is the **primary artifact**; code is merely its expression
- Specs are technology-independent – switching React → Vue doesn't change the spec
- Specs catch bugs **before they exist** – problems discovered before any code is written
- The spec is the **Single Source of Truth**

Section 1.2: SDD Principles & the 6 Phases

The Power Inversion – SDD's Core Insight

For decades, code was king. Specs served code.
SDD inverts this: code serves specifications.

- Specs don't merely guide implementation – they **generate** it
- The gap between spec and implementation is **eliminated**, not narrowed
- "Intent-driven development" – the lingua franca moves to a higher level
- This transforms the traditional SDLC – requirements and design become **continuous activities**, not discrete phases

Debugging means fixing specifications. Refactoring means restructuring for clarity.

Section 1.2: SDD Principles & the 6 Phases

The 6 Phases of SDD

```
1. Constitution  →  Define project principles and constraints
2. Specify       →  Describe what to build (not how)
3. Clarify       →  Surface hidden assumptions and edge cases
4. Plan          →  Choose architecture and technology
5. Tasks         →  Break down into executable steps
6. Implement     →  Generate code from specifications
```

Each phase produces an artifact. Each artifact references the previous one.

Section 1.2: Phase 1

Phase 1: Constitution

"Define project principles and constraints" – the project's DNA

Without a constitution, AI makes its **own assumptions** about:

- Test coverage requirements
- External dependency policy
- Error handling philosophy
- Code style and architecture patterns

Example Principles

- "Test coverage ≥ 80%"
- "Minimize external dependencies"
- "Fail-closed on authentication errors"

Section 1.2: Phase 2

Phase 2: Specify

"Describe what to build (not how)"

- User stories with acceptance criteria
- Intentional `[NEEDS CLARIFICATION]` markers for ambiguous points
- Technology-independent – no tech stack decisions here

Example

User Story: "As a team member, I can create retrospective cards."
Acceptance: Card has text (max 280 chars), author, timestamp.
`[NEEDS CLARIFICATION]`: Can users edit others' cards?

Section 1.2: Phase 3

Phase 3: Clarify ⭐ SDD's True Value

"Surface hidden assumptions and edge cases"

This is where SDD delivers the greatest ROI.
Catches ambiguity before a single line of code is written.

Questions that Clarify surfaces:

- "Can users vote on their own posts?"
- "What happens on network timeout?"
- "Are votes anonymous or visible?"
- "What's the maximum number of cards per session?"

All of these are things AI decides on its own without specs.

Section 1.2: Phase 4

Phase 4: Plan

"Choose architecture and technology" – technology discussion happens only now

- Separating "what" from "how" means specs survive technology changes
- Example: Switching React → Vue doesn't change the spec
- Research agents can investigate library compatibility, performance benchmarks, and security implications
- Organizational constraints integrate automatically

> Technology choices are **informed by the spec**, not the other way around.

Section 1.2: Phase 5

Phase 5: Tasks

"Break down into executable steps"

Each task has 4 essential elements:

1. **Clear scope** – exactly what to implement
2. **Traceable origin** – linked to user story in the spec
3. **Explicit ordering** – dependencies determine sequence
4. **Validation criteria** – how to verify completion

> Tasks are generated FROM the spec – not invented independently.

Section 1.2: Phase 6

Phase 6: Implement

"Generate code from specifications"

- AI follows `tasks.md` ordering
- References specs while writing code
- Does **NOT** add features not in the spec

Implementation observation points

- Does the generated code match the spec?
- Are permission rules from the Clarify phase respected?
- Does the test coverage meet the Constitution's requirements?

Section 1.2: Development Modes

3 Development Modes

1. 0-to-1 (Greenfield)

Generate from scratch – full SDD workflow, all 6 phases

2. Creative Exploration

Parallel implementations – experiment with diverse solutions from the same spec

3. Iterative Enhancement (Brownfield)

Add features to existing projects, modernize legacy codebases

> SDD works for new projects, experimentation, AND existing code.

Section 1.3: GitHub Spec Kit

What Is GitHub Spec Kit?

GitHub's open-source SDD toolkit

> "Build high-quality software faster."

Two pillars:

1. `specify` **CLI** – project setup, validation, versioning
2. **Copilot Chat** `/speckit.*` **slash commands** – execute each SDD phase

Supports 20+ AI agents – Copilot, Claude Code, Gemini CLI, Cursor, Codex, Windsurf, and more

Section 1.3: GitHub Spec Kit

Spec Kit's Two Pillars

	Pillar 1: `specify` CLI	Pillar 2: AI Instructions
Language	Python (installed via `uv`)	Markdown
Function	Project init, validation, versioning	SDD phase execution
Files	`specify init`, `check`, `version`	`.github/prompts/` + `.github/agents/`
Runtime	Terminal	Copilot Chat / AI Agent

CLI scaffolds the project. AI instructions execute each SDD phase.

Section 1.3: GitHub Spec Kit

Spec Kit Version Info

Component	Details
CLI Version	Installed via `uv tool install`
Template Version	Bundled with CLI, pinned per release
Runtime	Python 3.11+
Dev Container	Auto-pins version – no manual management

For local use: `specify version` shows CLI and template versions

Section 1.3: GitHub Spec Kit

Community & Adoption – 20+ Supported AI Agents

Agent	Agent	Agent	Agent
Copilot	Claude Code	Gemini CLI	Cursor
Codex	Windsurf	Qwen	opencode
Kilocode	Auggie	Roo	CodeBuddy
Amp	SHAI	Q	Bob
Qoder	Antigravity (agy)	Jules	**Generic**

Generic mode (`--ai generic`): Bring your own agent – SDD is a process, not tied to any tool

Section 1.3: GitHub Spec Kit

📽️ Video Overview – See SDD in Action

Official Spec Kit team video – full SDD walkthrough on a real project

📎 `youtube.com/watch?v=a9eR1xsfvHg`

Viewing Timing	Effect
After this presentation	Theory → visual reinforcement
Before hands-on	Grasp the full flow before starting
Review	Reconfirm phase connections

Theory → Video → Hands-on – 3 steps to master SDD

Section 1.3: GitHub Spec Kit

Spec Kit Architecture Overview

```
┌──────────────┐
│  specify CLI  │  ← Install & initialize
└──────┬───────┘
       │ specify init
       ▼
┌──────────────┬──────────────┬──────────────┐
│  .specify/   │  .github/    │  .vscode/    │  ← Project scaffolding
└──────┬───────┴──────┬───────┴──────┬───────┘
  Templates     Agents &      VS Code
  & Scripts     Prompts       Settings
      │             │
      ▼             ▼
┌──────────────────────┐
│  SDD Artifacts       │  ← AI generates
│  constitution / spec /  │
│  plan / tasks / code    │
└──────────────────────┘
```

Section 1.4: `specify` CLI

Installing the `specify` CLI

Method 1: Dev Container (Recommended)

```
# Dev Container / GitHub Codespaces — pre-installed
# Just start the container and you're ready
```

Method 2: Local Installation

```
# Prerequisites: Python 3.11+, uv, git, Node.js LTS
uv tool install specify-cli \
  --from git+https://github.com/github/spec-kit.git
```

Verify installation: `specify version`

Section 1.4: `specify` CLI

3 CLI Commands

Command	Purpose
`specify init`	Initialize project – download templates + generate directories
`specify check`	Verify required tool installations (git, AI tools, editor)
`specify version`	Show CLI/template version + system info

All three are terminal commands – NOT Copilot Chat commands

Section 1.4: `specify` CLI

`specify init` – Options in Detail

```
# Create new project directory
specify init my-project --ai copilot

# Initialize existing directory
specify init . --ai copilot --force
```

`--ai` options (20+)

`copilot` · `claude` · `gemini` · `cursor-agent` · `codex` · `windsurf` · `qwen` · `opencode` · `kilocode` · `auggie` · `roo` · `codebuddy` · `amp` · `shai` · `q` · `bob` · `qodercli` · `agy` · `generic`

Additional flags: `--ai-skills` (SKILL.md templates) · `--script ps` (Windows) · `--no-git` (skip Git) · `SPECIFY_FEATURE` env var (non-Git repos)

Section 1.4: `specify` CLI

`--ai generic` – Bring Your Own Agent

```
specify init my-project --ai generic \
  --ai-commands-dir .myagent/commands/
```

	Normal Agent	Generic
`specify check`	Validates agent installed	No check needed
Directory	Agent-specific path	Custom `--ai-commands-dir`
Configuration	Pre-defined	None required

SDD is a process, not tied to any specific tool – technology independence proven

Section 1.4: `specify` CLI

`specify check` – Environment Diagnostics

```
$ specify check

● Git                (available)
○ GitHub Copilot     (IDE-based – cannot check from CLI)
● Claude Code        (not found)
● Node.js            (v20.11.0)
● Python             (3.12.3)
```

See at a glance what's available.

Troubleshoot: `uv tool install specify-cli --force`

Section 1.5: Directory Structure

Project Structure After `specify init`

```
my-project/
├── .specify/          ← Scripts & templates
│   ├── scripts/
│   └── templates/
├── .github/           ← AI agents & prompts
│   ├── agents/
│   └── prompts/
├── .vscode/           ← VS Code settings
│   └── settings.json
└── constitution.md    ← Your project's DNA
```

Three directories form Spec Kit's scaffolding

Section 1.5: Directory Structure

`.specify/` Directory – Scripts & Templates

`scripts/bash/`

- `check-prerequisites.sh` – verify tool installations
- `common.sh` – shared utilities
- `create-new-feature.sh` – scaffold a new feature
- `setup-plan.sh` – initialize planning artifacts
- `update-agent-context.sh` – refresh agent context

`templates/`

- `constitution-template.md` · `spec-template.md`
- `plan-template.md` · `tasks-template.md`
- `checklist-template.md` · `agent-file-template.md`

Templates are the blueprints for SDD artifacts

Section 1.5: Directory Structure

`.github/` Directory – AI Agents & Prompts

Phase	Agent File	Prompt File
Constitution	`speckit.constitution.agent.md`	`speckit.constitution.prompt.md`
Specify	`speckit.specify.agent.md`	`speckit.specify.prompt.md`
Clarify	`speckit.clarify.agent.md`	`speckit.clarify.prompt.md`
Plan	`speckit.plan.agent.md`	`speckit.plan.prompt.md`
Tasks	`speckit.tasks.agent.md`	`speckit.tasks.prompt.md`
Implement	`speckit.implement.agent.md`	`speckit.implement.prompt.md`

`agent.md` = agent mode · `prompt.md` = slash command

Section 1.5: Directory Structure

`.vscode/settings.json` – VS Code Integration

```
{
  "chat.promptFilesRecommendations": [
    "speckit.constitution",
    "speckit.specify",
    "speckit.plan",
    "speckit.tasks",
    "speckit.implement"
  ]
}
```

Just open VS Code and `/speckit.*` commands are ready in Copilot Chat

- Auto-recommends Spec Kit commands in Copilot Chat
- Auto-approves `.specify/scripts/` terminal execution

Section 1.6: Command System

Core Commands (6) – Mapped to the 6 SDD Phases

#	Command	Purpose
1	`/speckit.constitution`	Generate constitution (project principles)
2	`/speckit.specify`	Define spec + `[NEEDS CLARIFICATION]` markers
3	`/speckit.clarify`	Resolve ambiguities in the spec
4	`/speckit.plan`	Architecture + data model
5	`/speckit.tasks`	Task breakdown + ordering
6	`/speckit.implement`	Generate code from specs

Each command runs in Copilot Chat – not the terminal

Section 1.6: Command System

Optional Commands (3) – Quality Enhancement

Command	When to Use	Purpose
`/speckit.clarify`	Before Plan	Resolve ambiguity proactively
`/speckit.analyze`	After Tasks	Cross-artifact consistency check
`/speckit.checklist`	After Plan	Generate quality checklist

Core commands are required; optional commands enhance quality.

- Special: `/speckit.taskstoissues` converts `tasks.md` to GitHub Issues

Section 1.6: Command System

Recommended Execution Flow

```
/speckit.constitution
    ↓
/speckit.specify
    ↓
(opt) /speckit.clarify
    ↓
/speckit.plan  →  (opt) /speckit.checklist
    ↓
/speckit.tasks  →  (opt) /speckit.analyze
    ↓
/speckit.implement
```

Special: `/speckit.taskstoissues` – convert tasks.md to GitHub Issues at any time

Section 1.6: Command System

3 Modes – prompt.md / agent.md / SKILL.md

Mode	File Type	Usage
Slash Command	`*.prompt.md`	`/speckit.specify` in Copilot Chat
Agent Mode	`*.agent.md`	Long-running tasks in Copilot Agent
Skills	`SKILL.md`	Install as agent skills (`--ai-skills`)

Same SDD phase, three execution modes.
This workshop uses slash commands (`prompt.md`).

Section 1.7: SDD Artifacts

SDD Artifact Overview & Relationship Diagram

```
constitution.md  →  spec.md  →  clarifications
      ↓                ↓              ↓
   Principles      What to       Resolved
   & constraints   build         ambiguities
                                     ↓
                    plan.md + data-model.md  →  tasks.md  →  Code
                       ↓                            ↓
                    How to                     Step-by-step
                    build                      ordering
```

Each artifact references the previous one – a chain of traceability

Section 1.7: SDD Artifacts

Template Contents – Constitution Template Example

```
## Core Principles

### [PRINCIPLE_NAME]
[PRINCIPLE_DESCRIPTION]

### [PRINCIPLE_NAME]
[PRINCIPLE_DESCRIPTION]

## Technology Constraints

### [CONSTRAINT_NAME]
[CONSTRAINT_DESCRIPTION]
```

AI fills in `[PLACEHOLDERS]` following the template structure.

Templates ensure quality and consistency across all projects.

Section 1.7: SDD Artifacts

The Constitution's 9 Articles – Immutable Principles

Based on `spec-driven.md`:

1. **Simplicity gate** – no over-engineering
2. **Library-first** – use existing solutions
3. **Pure-function core** – predictable business logic
4. **Observability via CLI** – debuggable from the terminal
5. **Single source of truth** – one canonical location per fact

And 4 more: test-first, modular by default, config over code, security by default

4 Design Philosophies: Modularity > Monoliths · Observability > Opacity · Simplicity > Cleverness · Integration > Isolation – these are **immutable**

Section 1.7: SDD Artifacts

7 Template Patterns That Constrain the LLM

#	Pattern	Effect
1	**What/How separation**	No tech stack in Specify phase
2	`[NEEDS CLARIFICATION]`	No guessing on ambiguity
3	**Checklists**	No skipping self-review
4	**Phase -1 Gates**	No over-complex designs
5	**Hierarchical detail**	No code in primary documents
6	**Test-first ordering**	No implementation before tests

Templates = "unit tests for specifications"

Pattern 7: No speculative features – no adding "nice to haves"

Section 1.7: SDD Artifacts

SDD Artifact Flow – Live Demo

```
① specify init . --ai copilot
    ↓
② Copilot Chat: /speckit.constitution
    ↓
③ constitution.md generated
    ↓
④ /speckit.specify  →  ⑤ spec.md generated
    ↓
⑥ /speckit.plan     →  ⑦ plan.md generated
    ↓
⑧ /speckit.implement  →  ⑨ Code generated from specs
```

Each step reads the previous artifact to generate the next

Section 1.8: Constitution Deep-Dive

Phase -1 Gates — Pre-Implementation Quality Checks

```
### Phase -1: Pre-Implementation Gates

#### Simplicity Gate (Article VII)
- [ ] Can this be solved with a single function?
- [ ] Does this need a new abstraction?

#### Anti-Abstraction Gate (Article VIII)
- [ ] Is a new interface justified?
- [ ] Would a concrete implementation suffice?
```

If a gate **fails** → document justification in "Complexity Tracking"

Gates act as compile-time checks for architectural principles

Section 1.8: Constitution Deep-Dive

The Compound Effect – Constraints Working Together

Quality	Enabled By
Complete	Checklists + no speculative features
Unambiguous	`[NEEDS CLARIFICATION]` markers
Testable	Test-first ordering + Phase -1 gates
Maintainable	Hierarchical detail + What/How separation
Implementable	Phase -1 gates + clear phase definitions

7 constraint patterns don't work individually – they **combine**
LLM transforms: **Creative Writer → Disciplined Specification Engineer**

Section 1.8: Constitution Deep-Dive

Concrete Example: Traditional ~12h vs SDD 15min

Chat feature – from requirements to implementation

Phase	Traditional	SDD
Requirements	PRD manual writing (2–3h)	`/speckit.specify` (5 min)
Technical Design	Design doc (3–4h)	`/speckit.plan` (5 min)
Test Planning	Manual scenarios (2h)	`/speckit.tasks` (5 min)
Total	**~12 hours**	**~15 minutes**

Output: `spec.md` + `plan.md` + `data-model.md` + `tasks.md`

Not just faster – templates enforce consistency and completeness

Section 1.9: Extension System

Extension System – Modular Extensibility

Command	Purpose
`specify extension add <repo>`	Install extension
`specify extension remove <name>`	Uninstall extension
`specify extension list`	Show installed extensions
`specify extension search <query>`	Search extension catalog
`specify extension update`	Update extensions
`specify extension enable/disable`	Toggle extensions

Customize and extend the SDD workflow for your team

Section 1.9: Extension System

Extension Manifest & Organization Customization

```yaml
extension:
  name: spec-kit-jira
  version: "0.1.0"
provides:
  commands:
    - name: jira-sync
      script: jira-sync.sh
hooks:
  after_tasks:
    - sync-to-jira.sh
```

- Custom catalog URL (`SPECKIT_CATALOG_URL`)
- Hooks for automation after SDD phases
- Distribute via GitHub repo tags → whole team

Section 1.9: Extension System

Example: Jira Integration Extension

```
specify extension add github.com/mbachorik/spec-kit-jira
```

What it does

- Syncs `tasks.md` items to Jira issues (like `/speckit.taskstoissues`)
- Maps SDD phases to Jira workflow states
- Integrates with existing project management tools

SDD doesn't replace your workflow – it enhances it

Section 1.10: Vibe Coding vs SDD

Vibe Coding vs SDD – Detailed Comparison

Axis	Vibe Coding	SDD
Starting point	Vague prompt	Structured specification
Edge cases	Discovered in production	Caught in Clarify phase
Documentation	None (or outdated)	Specs ARE the documentation
Pivot cost	Full rewrite	Spec update → regenerate
Where intent lives	Buried in code	Explicit in specs
AI output quality	🎲 Unpredictable	✅ Predictable

Section 1.10: Vibe Coding vs SDD

"Is SDD Slow?"

Time writing specs < Time fixing bugs

- Bugs caught in Clarify are bugs that **never exist**
- Pivoting is low-cost – specs are reusable across tech stacks
- The spec survives technology changes – code doesn't
- 3 months later: "Why this design?" → read the spec (not reverse-engineer the code)

"Writing correctly IS true speed"

Section 1.10: Vibe Coding vs SDD

Where SDD Shines (and Where It's Overkill)

✅ Best for:

- Team development (shared understanding through specs)
- AI-assisted development (AI follows structured specs)
- External service integrations (contracts prevent misunderstandings)
- Financial/security-critical systems (specs enforce correctness)
- Long-term maintenance projects (specs = living documentation)

⚠️ Overkill for:

- Throwaway scripts
- Exploratory prototypes (but move to SDD once you have direction)

Section 1.10: Vibe Coding vs SDD

Bidirectional Feedback – Specs ↔️ Production

- Production metrics and incidents don't just trigger hotfixes – they **update specifications**
- Performance bottlenecks become new non-functional requirements
- Security vulnerabilities become constraints for all future generations

Continuous evolution, not one-shot generation

SDD transforms the traditional SDLC:

- Requirements and design become **continuous activities**
- Not discrete phases that happen once and are forgotten

Section 1.10: Vibe Coding vs SDD

Branching for Exploration

SDD supports **parallel implementations** from the same spec:

- Different tech stacks (React vs Vue vs Svelte)
- Different architectures (monolith vs microservices)
- Different UX patterns – all generated from ONE specification

```
0 → 1, (1', 1'', …), 2, 3, N
```

Team process

- Specs are versioned in branches, reviewed, and merged
- `specs/[branch]/` for team review workflows

Branching isn't just for code – it's for exploring design alternatives

Chapter 2: Beginner – SDD Workflow Fundamentals

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation

Section 2.1: Introduction

Beginner Scenarios

Scenario	Build	SDD Skills	Time
A: QuickRetro	Retrospective board	Data modeling, CRUD, permissions	~90 min
J: Pomodoro	Timer app	State machine specification	~60 min
K: MarkdownPad	Markdown editor	Output correctness, security	~60 min
L: RecipeBox	Recipe manager	Calculation correctness	~60 min

No server, no database. **Learn the SDD workflow itself.**

Section 2.1: Introduction

Beginner Tech Stack

- **HTML / CSS / JavaScript** – vanilla, no frameworks
- **localStorage** – client-side persistence
- **No build tools** – no webpack, no bundler
- **No server** – everything runs in the browser

Why? **Eliminate technical complexity to focus on learning SDD.**
The workflow is the same regardless of tech stack.

Section 2.2: QuickRetro Walkthrough

Scenario A: QuickRetro

Build a team retrospective board

- Create/edit/delete retrospective cards
- Fixed user selection (4 team members)
- 3 columns: Good / Improve / Action Items
- Voting system with rules

SDD Skills

Data modeling, CRUD specifications, permission matrix

Duration: ~90 minutes

Section 2.2: QuickRetro Walkthrough

Constitution in Practice

Example Principles (QuickRetro)

1. **Simplicity** – vanilla JS, no frameworks
2. **Readability** – code should be self-documenting
3. **Minimal dependencies** – no external libraries
4. **Test coverage** – core logic must be testable
5. **Accessible UI** – basic ARIA compliance

Key point: principles are short in proportion to risk.
Beginner project = 4–5 principles. Enterprise = 11+.

Section 2.2: QuickRetro Walkthrough

Specify in Practice – Intentional Ambiguity

Spec contents:

- Fixed user selection (4 users)
- 3 fixed columns (Good / Improve / Action Items)
- Card CRUD operations
- Voting rules

`[NEEDS CLARIFICATION]` **markers appear:**

- Can anyone create a session, or only admins?
- Can users edit **other people's** cards?
- Can you vote on your **own** card?

> The spec **intentionally leaves ambiguities** for the Clarify phase

Section 2.2: QuickRetro Walkthrough

Clarify in Practice – 10 Hidden Assumptions

- "Can anyone create a session, or only admins?"
- "Can you vote on your own card?"
- "Are votes anonymous or visible?"
- "Can users change their vote?"
- "What's the maximum number of cards per session?"

All of these are things AI decides on its own without specs.

If you don't specify → AI silently picks an answer.
Both "yes" and "no" are reasonable. But **only the user knows the intent.**

Section 2.2: QuickRetro Walkthrough

The Clarify "Aha Moment"

"Can you vote on your own card?"

Without specs, AI silently picks "yes" or "no."

- **If "yes"** → users can inflate their own card importance
- **If "no"** → voting reflects genuine team consensus

Both designs are reasonable. But the business intent is completely different.

User intent can only be communicated through specifications.
This single question illustrates SDD's entire value proposition.

Section 2.2: QuickRetro Walkthrough

Plan → Tasks → Implement

Plan

Vanilla HTML/CSS/JS + localStorage → no build step, no server

Tasks (ordering matters)

1. Data model (Card, User, Vote, Session)
2. Service layer (CRUD + permission checks)
3. UI components (card rendering, voting, column layout)

Implement

- AI follows task ordering
- Permission checks are centrally managed (not scattered in click handlers)

3 phases condensed – but each reads the previous artifact

Section 2.2: QuickRetro Walkthrough

Permission Matrix

Action	Card Author	Other Users	Session Admin
Create card	✅	✅	✅
Edit card	✅	❌	✅
Delete card	✅	❌	✅
Vote on card	❌ (own)	✅	✅
Create session	❌	❌	✅

This matrix doesn't get generated without a spec.
Without SDD, AI decides all of these silently.

Section 2.2: QuickRetro Walkthrough

QuickRetro Takeaways

1. **Completed the full 6-phase SDD workflow** – Constitution through Implement
2. **Permission rules must be explicitly specified** – AI cannot infer business intent
3. **Clarify has the highest ROI** – catches decisions before code exists
4. **Without specs, AI decides your design for you** – silently, without asking

Chapter 2's key message: **"Without specs, AI decides your design for you."**

Section 2.3: Other Beginner Scenarios

Scenario J: Pomodoro Timer — State Machine Specification

5 timer states: **Idle → Focus → Short Break → Long Break → Paused**

SDD Lesson

- **Behavior is specified using state machines**
- Every state transition must be explicitly defined
- Wall-clock vs `setInterval` drift problem → spec must address timing precision

> Without a state machine spec, AI invents its own transitions

Section 2.3: Other Beginner Scenarios

Scenario K: MarkdownPad – Output Correctness

The security question: What does the editor render?

- **Whitelist approach:** "If it's not on the list, don't render it"
- **Blacklist approach:** "Block known-dangerous tags" (vulnerable to new attacks)

SDD Lesson

- **Output specs are directly tied to security (XSS prevention)**
- URL scheme restrictions: only `http:`, `https:`, `mailto:`
- `javascript:` URLs must be explicitly blocked in the spec

> Without output specs, AI renders everything – including `<script>` tags

Section 2.3: Other Beginner Scenarios

Scenario L: RecipeBox – Calculation Correctness

The "1.5 eggs" problem

```
// ❌ parseFloat approach
Math.round(parseFloat('1.005') * 100) // → 100, not 101!

// ✅ Fraction arithmetic
{ numerator: 3, denominator: 2 } // 1.5 — exact representation
```

SDD Lesson

- **Calculation specs must state precision requirements explicitly**
- `{ numerator, denominator }` representation avoids floating-point errors
- Rounding rules must be in the spec (round up? truncate? banker's rounding?)

Section 2.3: Other Beginner Scenarios

Beginner Progression – Skills That Build

Order	Scenario	New SDD Skill
1st	**A: QuickRetro**	Data modeling + CRUD + permissions
2nd	**J: Pomodoro**	State machine specification
3rd	**K: MarkdownPad**	Output correctness + security
4th	**L: RecipeBox**	Calculation correctness

Each step adds a **new specification skill** – not just a new app

Section 2.4: Beginner Summary

4 Core Beginner Lessons

1. **Permission specs** – Who can do what → Scenario A
2. **Behavior specs** – What are the state transitions → Scenario J
3. **Output specs** – What to render (and what NOT to) → Scenario K
4. **Calculation specs** – What are the precision requirements → Scenario L

> 4 types of specification. 4 types of bugs prevented.

Section 2.4: Beginner Summary

Common Beginner Pitfalls

1. Permission checks **scattered across click handlers** (not centralized)
2. State machines become `if/else` **chains** (not explicit state transitions)
3. **"Render everything"** instead of whitelisting (XSS vulnerability)
4. `parseFloat` for fraction arithmetic (precision errors)

> Every pitfall is a **missing specification** that AI fills with its own assumptions

Section 2.4: Beginner Summary

Bridge to Intermediate

Beginner: client-side only → localStorage → no API

Intermediate: a server enters the picture

- API contracts become the center of your specs
- HTTP semantics matter (status codes, methods, headers)
- Databases replace localStorage
- Multiple consumers (browser, mobile, CLI) hit the same API

How do specs change when there's a server? → **Chapter 3**

Chapter 3: Intermediate — Server-Side & API Contracts

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation

Section 3.1: Introduction

Intermediate Scenarios

Scenario	Build	SDD Skills
H: CLI	Cross-platform CLI tool	Output contract, exit codes
M: ShortLink	URL shortener + analytics	API contract, HTTP semantics
N: KanbanFlow	Kanban board	Ordering algorithms, intent-based API
O: MoneyTrail	CSV expense importer	Data validation pipeline

Add a server. **API contracts become the center of your specs.**

Section 3.1: Introduction

Baseline Contract – Shared by All ⭐⭐ Scenarios

All intermediate scenarios share these conventions:

- **Standard error envelope** – JSON `{ error: { code, message } }`
- **Cursor pagination** – `?cursor=<token>&limit=20`
- **ISO 8601 dates** – `2026-02-25T21:36:37Z`
- **DB conventions** – `PRAGMA foreign_keys = ON`, `id` / `createdAt` / `updatedAt`
- **Standard scripts** – `npm start` / `npm test`

A shared baseline means each scenario focuses on its unique SDD lesson

Section 3.2: ShortLink Walkthrough

Scenario M: ShortLink – URL Shortener + Analytics

Looks simple. But when you write the spec, questions emerge:

- 301 vs 302 redirect?
- Route ordering in Express?
- What happens to deleted short URLs?
- Shorten the same URL twice – new slug or existing?

SDD lesson: **The API contract IS the product**

Section 3.2: ShortLink Walkthrough

HTTP Status Codes Are Part of the Spec

	301 Permanent	302 Temporary
Browser behavior	Caches redirect	Hits server each time
Analytics	❌ Stops counting	✅ Every click recorded
SEO	Transfers link equity	Preserves original URL

Status code selection is a business decision, not a technical one.
Without specs, AI picks one – your analytics may break silently.

Section 3.2: ShortLink Walkthrough

Route Safety – Express Mount Order

```
// ❌ Wrong order – /:slug catches EVERYTHING
app.get('/:slug', redirectHandler);
app.get('/api/health', healthHandler);   // Never reached!

// ✅ Correct order – mount /api/* first
app.get('/api/health', healthHandler);
app.get('/api/shorten', shortenHandler);
app.get('/:slug', redirectHandler);      // Last = catch-all
```

Route ordering must be explicitly specified in the API contract

Section 3.2: ShortLink Walkthrough

Idempotency – Shorten the Same URL Twice?

Option A: Generate a new slug

- Response: `201 Created` with a new short URL
- Same long URL → multiple short URLs

Option B: Return existing active link

- Response: `200 OK` with the existing short URL
- Same long URL → always the same short URL

SDD resolves this in Clarify.
Without specs, AI picks one arbitrarily.

Section 3.2: ShortLink Walkthrough

Deletion Semantics – 404 vs 410

	404 Not Found	410 Gone
Meaning	"Never existed"	"Once existed, now deleted"
Slug reuse	Slug can be reused	Slug permanently reserved
Guarantee	None	Slug permanence

Recommended: `410 Gone` + tombstone record

Deletion semantics are business rules, not HTTP defaults.
The slug `abc123` was shared on Twitter – it must never point somewhere else.

Section 3.2: ShortLink Walkthrough

Error Format Split

Consumer	Endpoint	Error Format
Browser	`GET /:slug`	HTML error page
API client	`POST /api/shorten`	JSON error envelope
API client	`GET /api/stats/:slug`	JSON error envelope

Same app, different consumers = different error formats.
The spec must define error format per endpoint, not globally.

Section 3.2: ShortLink Walkthrough

ShortLink Takeaways

1. **HTTP status codes are part of the spec** – 301 vs 302 is a business decision
2. **Route ordering must be specified** – `/:slug` catches everything
3. **Idempotency is resolved in Clarify** – not left to AI
4. **Deletion semantics are business rules** – 404 vs 410
5. **Error format varies by consumer** – HTML for browsers, JSON for APIs

> What looks "simple" has at least **5 specification decisions** hiding beneath the surface

Section 3.3: Other Intermediate Scenarios

Scenario H: CLI – Output Contract

The CLI's "screen" is **stdout** – output itself IS the contract

- **stderr / stdout separation** – errors go to stderr, data to stdout
- **Exit codes** – `0` success, `1` general error, `2` usage error
- **JSON output mode** – `--json` flag for machine-readable output
- Piping: `mycli list | jq '.[] | .name'` must work

SDD lesson: **Non-web software has contracts too.**
Every output format decision belongs in the spec.

Section 3.3: Other Intermediate Scenarios

Scenario N: KanbanFlow – Fractional Indexing

Card ordering is **data** – not just UI presentation

- **Fractional indexing:** assign numeric positions between existing cards
- **Intent-based API:** client says "move card between A and B" → server calculates position
- No sequential integer IDs for order (reordering becomes expensive)

SDD lesson: **Ordering algorithms are part of the spec.**
Without specs, AI picks array splice or sequential IDs – both break at scale.

Section 3.3: Other Intermediate Scenarios

Scenario O: MoneyTrail – Data Validation Pipeline

Each CSV row gets a **verdict**: success / warning / error / skipped

```
// ❌ The parseFloat trap
Math.round(parseFloat('1.005') * 100) // → 100, not 101!

// ✅ parseCents() — string splitting
'1.005'.split('.') → parse integer parts → 1005 cents
```

SDD lesson: **Data quality specs design the validation pipeline.**
Every row verdict must be defined in the spec.

Section 3.4: Intermediate Summary

Intermediate Progression

Order	Scenario	New Server-Side SDD Skill
1st	**H: CLI**	Output contract (stdout/stderr/exit codes)
2nd	**M: ShortLink**	API contract (HTTP semantics)
3rd	**N: KanbanFlow**	Ordering algorithm specification
4th	**O: MoneyTrail**	Data validation pipeline

Each step adds a new **server-side specification skill**

Section 3.4: Intermediate Summary

Beginner vs Intermediate – What Changed?

Axis	⭐ Beginner	⭐⭐ Intermediate
Tech stack	localStorage	SQLite + Express
Spec center	UI behavior	API contract
Testing	Manual	supertest + automated
Constitution	4–5 principles	6 principles
Consumer	Self (browser)	External clients

The spec focus shifts from **UI behavior** to **API contracts**

Section 3.4: Intermediate Summary

Bridge to Intermediate–Advanced

Intermediate: single service + own database

Intermediate–Advanced: external service integrations enter the picture

- OAuth / OIDC authentication providers
- Payment processors (Stripe)
- Infrastructure as Code (Terraform)
- API versioning for existing consumers

Where missing specs cause **real damage** – real money, real security breaches → **Chapter 4**

Chapter 4: Intermediate–Advanced — Real-World Complexity

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation

Section 4.1: Introduction

Intermediate–Advanced Scenarios

Scenario	Build	SDD Skills
B: PWA	Field inspection app	Offline sync, conflict resolution
C: OIDC SSO	SSO + RBAC	Fail-closed, tenant isolation
D: Stripe	Subscription billing	Monetary correctness, idempotency
G: Terraform	IaC pipeline	Drift detection, blast radius
I: API Versioning	v1→v2 migration	Backward compatibility

Where missing specs cause real damage.

Section 4.2: Stripe Walkthrough

Scenario D: Stripe Subscriptions

Subscription + dunning flow

SDD Skills

- Monetary correctness (integer cents)
- Idempotency (every API call)
- Webhook state machines (at-least-once delivery)

Billing bug = losing real money.
This is where specification rigor pays for itself.

Section 4.2: Stripe Walkthrough

Integer Cents — The Floating-Point Trap

```
// ❌ The trap
0.1 + 0.2           // → 0.30000000000000004
19.99 * 100          // → 1998.9999999999998

// ✅ Integer cents — always exact
1999                 // $19.99 stored as 1999 cents
1999 + 500           // $19.99 + $5.00 = $24.99 (2499 cents)
```

SDD makes this explicit in the constitution:
"monetary precision = integer cents. `parseFloat` is forbidden."

Section 4.2: Stripe Walkthrough

Complex State Machine — 9 Transitions

```
trial → active → past_due → canceled
  ↓        ↓         ↓
active   past_due   suspended
             ↓
           canceled → (end)
```

Evolution from beginner Pomodoro: 5 states → **8+ states, 9 transitions**

If you don't specify every state transition, users get incorrectly billed.
A missing transition = a billing bug in production.

Section 4.2: Stripe Walkthrough

Webhook Reliability – "At Least Once" Delivery

Stripe webhooks are delivered **at least once** – duplicates happen.

Must specify:

1. **Signature verification** – reject unsigned events
2. **Event deduplication** – track processed event IDs
3. **Async queue processing** – don't block the webhook endpoint
4. **Stale event handling** – ignore events older than current state

Duplicate processing = double billing.
Without specs, AI skips deduplication entirely.

Section 4.2: Stripe Walkthrough

Idempotency Keys — Every Stripe API Call

Without idempotency:

```
POST /charge → network error → retry → 💸 double charge!
```

With idempotency:

```
POST /charge (key: abc123) → error → retry (key: abc123) → ✅ same result
```

SDD makes **"idempotency everywhere"** a constitutional principle.
Every Stripe API call must include an idempotency key – no exceptions.

Section 4.2: Stripe Walkthrough

The Clarify "Aha Moment" – Unused Seat Refunds

"What about unused seat credits at cancellation?"

- **Refund** pro-rata to the customer?
- **Forfeit** – credits expire at cancellation?
- **Apply to final invoice** – credit against outstanding balance?

Each option has different financial, legal, and UX implications.

Without specs, AI decides your refund policy.
Financial edge cases must be explicitly decided – never left to AI.

Section 4.2: Stripe Walkthrough

Constitution Density Comparison

	★ Beginner	★★★ Stripe
Principles	4–5	11
Examples	Simplicity, readability	Correctness-first, integer cents
	Minimal deps, test coverage	Idempotency, eventual consistency
		Fail-safe access, webhook verification
		PCI compliance, audit logs

Constitution grows proportionally to risk.
More money at stake → more principles required.

Section 4.3: Other Scenarios

Scenario B: Field Inspection PWA – Offline Sync

Specifying offline-first behavior:

- **Conflict resolution rules** – last-write-wins? merge? user chooses?
- **Media upload retry** – resume interrupted uploads
- **Sync queue ordering** – which changes sync first?
- Queue persistence across app restarts

SDD lesson: **"What happens offline?" is answered in the spec.**
Without specs, offline behavior is undefined – data loss in the field.

Section 4.3: Other Scenarios

Scenario C: OIDC SSO + RBAC – Security Contract

- **Fail-closed principle** – on auth error, deny access (never fail-open)
- **Tenant isolation** – user A must NEVER see user B's data
- **Secret rotation** – IdP signing keys rotate; app must handle gracefully
- **RBAC matrix** – define roles × permissions × resources

SDD lesson: **Security isn't bolted on later – it's defined in the constitution.**
A missing `fail-closed` principle → cross-tenant data leak.

Section 4.3: Other Scenarios

Scenario G: Terraform + GitHub Actions – Infrastructure Spec

"Infrastructure itself IS the spec."

- **Drift detection** – alert when actual state ≠ desired state
- **Blast radius control** – limit what a single change can affect
- **No manual changes** – all changes through `terraform apply`
- **State locking** – prevent concurrent modifications

SDD lesson: **SDD applies to ops, not just apps.**
Infrastructure specifications prevent production incidents.

Section 4.3: Other Scenarios

Scenario I: API Versioning – Backward Compatibility

v1 → v2 migration with 200 existing consumers:

- **Shim layer** – v1 requests translated to v2 internally
- **Deprecation timeline** – v1 sunset date communicated 6 months ahead
- `410 Gone` – after sunset, v1 endpoints return 410 (not 404)
- Monitoring: track v1 usage to know when it's safe to remove

SDD lesson: **200 consumers depend on v1 – compatibility is a spec requirement.**
Breaking changes without a migration spec → customer churn.

Section 4.4: Summary

Intermediate vs Intermediate–Advanced – What Changed?

Axis	★★ Intermediate	★★★ Int–Advanced
External services	None	Stripe / IdP / Terraform
Stakes	UI bugs	Real money / security
Constitution	6 principles	11 principles
Consistency	Not needed	Eventual consistency
Testing	Contract tests	Chaos testing

Section 4.4: Summary

"Missing Specs Cause Real Damage" – Case Studies

1. 💸 **Double billing** – missing idempotency spec (Scenario D)
2. 🔓 **Cross-tenant data leak** – missing fail-closed principle (Scenario C)
3. 📘 **Offline data loss** – missing sync spec (Scenario B)
4. 💔 **Sudden API shutdown** – missing migration spec (Scenario I)
5. 🔥 **Infrastructure drift** – missing IaC spec (Scenario G)

All preventable with Clarify.
The cost of writing specs << the cost of these failures.

Section 4.4: Summary

Bridge to Advanced

Intermediate–Advanced: external service integrations

Advanced: distributed systems – failure is the default state

- Safety invariants vs liveness goals
- Compensating transactions (Sagas)
- Consistency models (CRDT, OT, authoritative server)
- "Timeout = unknown" paradigm

Specs must define the **failure model** – not just the happy path → **Chapter 5**

Chapter 5: Advanced — Distributed Systems & Failure Models

Shinya Yanagihara, Developer Global BlackBelt, Microsoft Corporation

Section 5.1: Introduction

Advanced Scenarios

Scenario	Build	SDD Skills
E: Whiteboard	Collaborative whiteboard	Consistency model, latency budget
F: Pipeline	Event ingestion pipeline	SLO specification, backpressure
P: OrderFlow	Order fulfillment saga	Compensating transactions, outbox
Q: PlugKit	Plugin runtime	Public API contract, sandboxing
R: FlagShip	Feature flag system	Statistical correctness, bucketing

Failure is the default state. Specs define the failure model.

Section 5.1: Introduction

Advanced Baseline Contract

Required for ALL ★★★★ scenarios:

1. **Failure model section** – enumerate what can fail and how
2. **Safety invariants vs liveness goals** – things that must NEVER happen vs things that must EVENTUALLY happen
3. **Idempotency rules** – per-operation idempotency guarantees
4. **Observability** – correlation IDs, audit logs, reconciliation endpoints

> Without a failure model, your spec only covers the happy path.
> **In distributed systems, the happy path is the exception.**

Section 5.2: Safety vs Liveness

Safety Invariants – Things That Must NEVER Happen

- **"Never ship an unpaid order"** (Scenario P: OrderFlow)
- **"Never assign a user to two mutually exclusive experiments"** (Scenario R: FlagShip)
- **"Deleted objects never appear to other users"** (Scenario E: Whiteboard)
- **"Never process the same event twice"** (Scenario F: Pipeline)

Explicitly enumerate safety invariants in the spec.
If AI doesn't know what must NEVER happen, it can't prevent it.

Section 5.2: Safety vs Liveness

Liveness Goals – Things That Must EVENTUALLY Happen

- **"All clients eventually converge"** (Scenario E: Whiteboard)
- **"Buffered events are eventually processed"** (Scenario F: Pipeline)
- **"Orders eventually complete or get compensated"** (Scenario P: OrderFlow)
- **"Feature flag changes eventually propagate"** (Scenario R: FlagShip)

"Eventually" is the keyword.
Liveness goals define the system's convergence guarantees.

Section 5.3: OrderFlow Walkthrough

Scenario P: OrderFlow Saga

6-step order fulfillment:

```
Validate → Authorize → Reserve → Capture → Ship → Confirm
```

SDD Skills

- Compensating transactions (undo each step)
- Timeout semantics ("timeout = unknown")
- Exactly-once processing guarantees

Duration: ~120 minutes (most complex scenario)

Section 5.3: OrderFlow Walkthrough

Compensation Pairs & the Cost Gradient

Step	Compensation	Cost
Validate	None	Free
Authorize	Void authorization	Free
Reserve	Release inventory	Free
Capture	Refund	💸 Fee + 5–10 days
Ship	Cancel shipment	⏰ Time-limited

The further you go, the more expensive compensation becomes.

Step ordering matters – compensate cheaply first.

Section 5.3: OrderFlow Walkthrough

"Timeout = Unknown" Paradigm

Payment capture times out. Was the customer charged?

Answer: Unknown.

- ❌ Retry immediately → potential **double charge**
- ❌ Assume failure → customer charged but order not fulfilled
- ✅ **Query status first**, then decide next action

In vibe coding, timeout becomes "retry → double charge."
SDD forces you to specify timeout behavior for every step.

Section 5.3: OrderFlow Walkthrough

Failure Paths Outnumber the Happy Path

One happy path. But:

- 6 steps × multiple failure modes
- Partial completion states
- Timeout at any step
- Network errors between services
- Concurrent saga instances

Without specs, it's impossible to discover all failure paths.
Combinatorial explosion makes "figure it out later" catastrophic.

Section 5.3: OrderFlow Walkthrough

The Outbox Pattern

Problem:

State changes in DB, but event NOT sent to message queue → inconsistency

Solution: Outbox Pattern

```
BEGIN TRANSACTION
  UPDATE orders SET status = 'captured'
  INSERT INTO outbox (event_type, payload) VALUES ('captured', ...)
COMMIT
-- Background worker reads outbox → publishes events
```

Specify event delivery guarantees in the spec.
"State change + event" must be atomic – no partial updates.

Section 5.3: OrderFlow Walkthrough

OrderFlow Takeaways

1. **Compensate cheaply first** – step ordering matters
2. **Timeout ≠ failure** – query status before retrying
3. **Failure paths are combinatorial** – specs enumerate them
4. **Safety invariant: never ship unpaid** – explicitly in spec
5. **Outbox pattern** – atomic state change + event publishing

> Saga specs are the most complex – but also where SDD provides the **greatest protection**

Section 5.4: Other Advanced Scenarios

Scenario E: Collaborative Whiteboard – Consistency Model

Consistency model choice determines **entire architecture**:

Model	Pros	Cons
CRDT	No central server	Complex merge logic
OT	Proven (Google Docs)	Requires transform functions
Authoritative	Simple	Single point of failure

Quantitative latency budget: local < 16ms, remote p95 < 200ms

SDD lesson: **Architecture decisions happen in the spec/plan phase**

Section 5.4: Other Advanced Scenarios

Scenario F: Event Ingestion Pipeline – SLO Specification

Ban vague "high performance." Specify concrete SLOs.

- **Throughput:** X events/sec sustained
- **Latency:** p99 < Y milliseconds
- **Backpressure:** return `429 Too Many Requests` when overloaded
- **Schema evolution:** backward-compatible event format changes

SDD lesson: **Performance requirements must be quantitatively specified.**
"Fast" is not a specification.

Section 5.4: Other Advanced Scenarios

Scenario Q: PlugKit Runtime – Your Spec IS the Product

The plugin API is what external developers depend on.

- **Sandboxed execution** – Worker Thread isolation
- **Capability permissions** – plugins declare what they need
- **Threat model** – malicious plugins can't escape sandbox
- **Versioning** – plugin API v1 must remain stable

SDD lesson: **Public API specs can't be changed after release.**
Your spec literally IS the product contract.

Section 5.4: Other Advanced Scenarios

Scenario R: FlagShip – Statistical Correctness

- **Deterministic bucketing** – `Math.random()` is **forbidden**
  - Use hash functions for reproducible assignment
- **SRM detection** – chi-square test for Sample Ratio Mismatch
- **Kill switch** – bypasses all caches, immediate flag override

SDD lesson: **Statistical correctness is part of the spec.**
A bucketing bug invalidates every experiment result.

Section 5.5: Advanced Summary

Advanced Progression

Order	Scenario	New Distributed Systems SDD Skill
1st	**E: Whiteboard**	Consistency model, latency budget
2nd	**F: Pipeline**	SLO quantification, backpressure
3rd	**P: OrderFlow**	Compensating transactions, outbox
4th	**Q: PlugKit**	Public API contract, sandboxing
5th	**R: FlagShip**	Statistical correctness, bucketing

Each step adds a new **distributed systems specification skill**

Section 5.5: Advanced Summary

Cross-Level Comparison — What Changed?

Axis	★ Beginner	★★ Intermediate	★★★ Int–Adv	★★★★ Advanced
Constitution	4–5	6	11	11+
State complexity	5 states	–	8+ states	Saga + CRDT
Failure model	None	API errors	Webhooks	Combinatorial
Idempotency	None	API level	Webhook + API	Per-step
"Aha moment"	Self-voting	301 vs 302	Float trap	Timeout=unknown

Section 5.5: Advanced Summary

SDD Skill Growth Map

State Machines

```
J (5 states) → D (8+ states, billing) → P (Saga, 6-step + compensation) → E (CRDT)
```

Permissions

```
A (matrix) → M (route safety) → D (PCI compliance) → E (object ownership)
```

Idempotency

```
None → API level (M) → Webhook (D) → Per-step key (P)
```

Consistency

```
localStorage → Single DB (M) → Eventual consistency (D) → Real-time convergence (E)
```

Chapter 6: Summary & Next Steps

Chapter 6

The Transformation – What SDD Changes

SDD doesn't replace developers – it automates mechanical translation and amplifies human capability.

Axis	Traditional	SDD
Central asset	Code (specs serve code)	Spec (code serves specs)
Maintenance	Fix code, docs get outdated	Evolve specs, regenerate code
Pivoting	Full rewrite (high cost)	Spec update → regenerate (low cost)
Developer role	Mechanical translation	Creativity, experimentation, critical thinking

The lingua franca of development moves from code to natural-language specifications.
Debugging means fixing specifications. Refactoring means restructuring for clarity.

Chapter 6

4 Core SDD Principles

1. Specs catch bugs before they exist

Bugs found in Clarify **never reach production**

2. AI works dramatically better with structure

Structured specifications = predictable AI output

3. Pivoting becomes low-cost

Update the spec → regenerate. No full rewrite needed.

4. The spec is the Single Source of Truth

3 months later: "Why this design?" → read the spec

Chapter 6

Recommended Learning Paths

Available Time	Recommended Path
90 minutes	Scenario A (QuickRetro) – full SDD workflow
Half day	A → M (ShortLink) – add API contracts
Full day	A → J → K → M → N → pick one ⭐⭐⭐
Advanced track	M → D → P → R – shortest to distributed SDD
Team training	Everyone starts with A → self-select by level
Conference (60 min)	A or J, MVP tier only

Tips: Never skip Clarify (ROI source) · Check answer keys **after**, not during · Watch constitution density grow with risk

Chapter 6

SDD Practice Guide – Start Today Checklist

#	Action	Key Point
1	`specify init` **your project**	Constitution + templates auto-generated
2	**Write constitution first**	Principles before specs – not after
3	**Never skip Clarify**	`[NEEDS CLARIFICATION]` is SDD's ROI source
4	**Manage specs in branches**	`specs/[branch]/` for team review & merge
5	**Start with one feature**	Experience specify → plan → tasks on one feature
6	**Verify AI output vs spec**	Always cross-check generated code

Mindset shift: Before writing code, ask "Is the spec sufficient?"

Chapter 6

Resources

Spec Kit

- **Repository:** `github.com/github/spec-kit` · **Docs:** `github.github.io/spec-kit/`
- 📽️ **Video:** `youtube.com/watch?v=a9eR1xsfvHg` · **Workshop:** `WORKSHOP.md` · **Scenarios:** `SCENARIOS.md`

SDD Methodology

- **Complete guide:** `spec-driven.md` · **Phase details:** each `/speckit.*` prompt file

Get Started Now

```
specify init my-first-sdd-project --ai copilot
cd my-first-sdd-project && code .  # then: /speckit.constitution
```

Stop vibe coding. Start specifying.